3/5/2025

Assignment-03

Course Title: Data Structure and Algorithms-II Lab
Course Code: CSE 2202

Prepared by:

Md. Ehasan Mahmud
Roll: 202122104026
Department: Computer Science and Engineering

Submitted to:

Dr. Sujan Chandra Roy
Lecturer
Department of Computer Science and Engineering
Kishoreganj University.

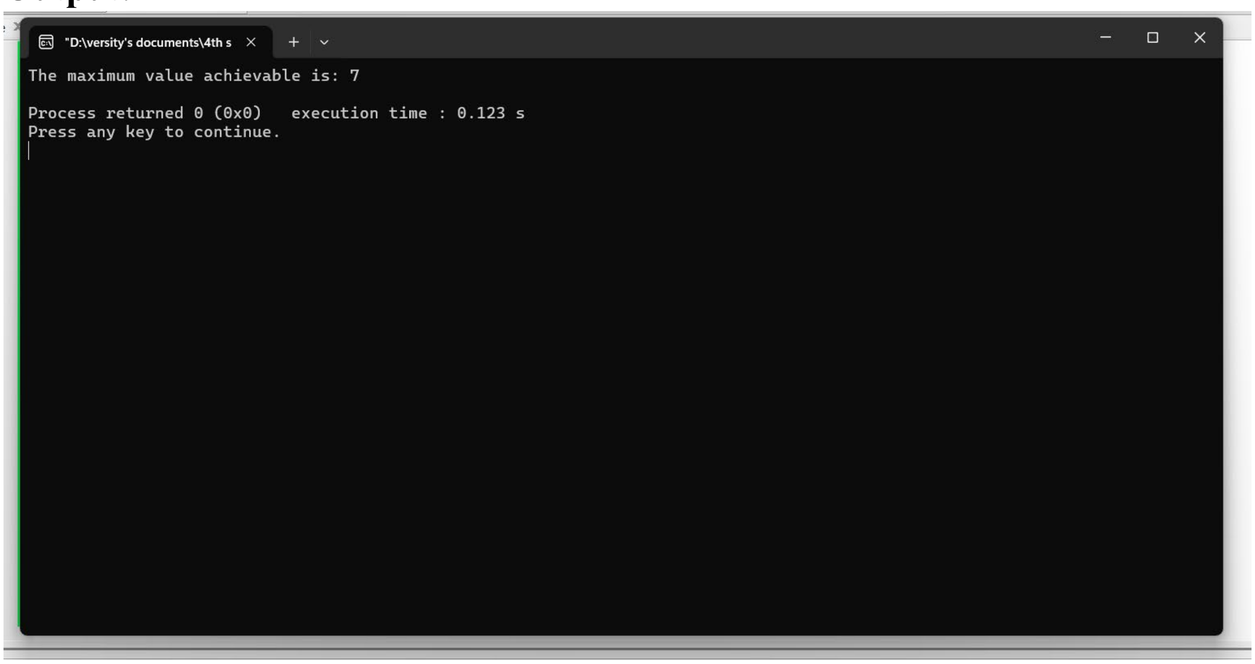
Demonstrate the 0-1 Knapsack problem solution:

Code:

```
#include <bits/stdc++.h>
using namespace std;
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
using ll=long long;
using ull=unsigned long long;
using ld=long double;
const ll MOD=998244353;
const ll M=200005;
template<typename T>
using indexed_set=tree<T,null_type,less<T>,rb_tree_tag,tree_order_statistics_node_update>;
ll calculateMaxValue(vector<ll>&weights,vector<ll>&values,ll capacity)
{
  ll numItems=values.size();
  vector<ll>dynamicProgramming(capacity+1,0);
  for(ll i=0; i<numItems; i++)
  {
    for(ll j=capacity; j>=weights[i]; j--)
    {
      dynamicProgramming[j]=max(dynamicProgramming[j],values[i]+dynamicProgramming[j-weights[i]]);
    }
  }
  return dynamicProgramming[capacity];
}
void executeSolution()
{
  vector<ll>weights= {2,3,4,5};
  vector<ll>values= {3,4,5,6};
```

```
    ll maxCapacity=5;
    ll result=calculateMaxValue(weights,values,maxCapacity);
    cout<<"The maximum value achievable is: "<<result<<endl;
}
int main()
{
    ios_base::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    executeSolution();
    return 0;
}
```

Output:



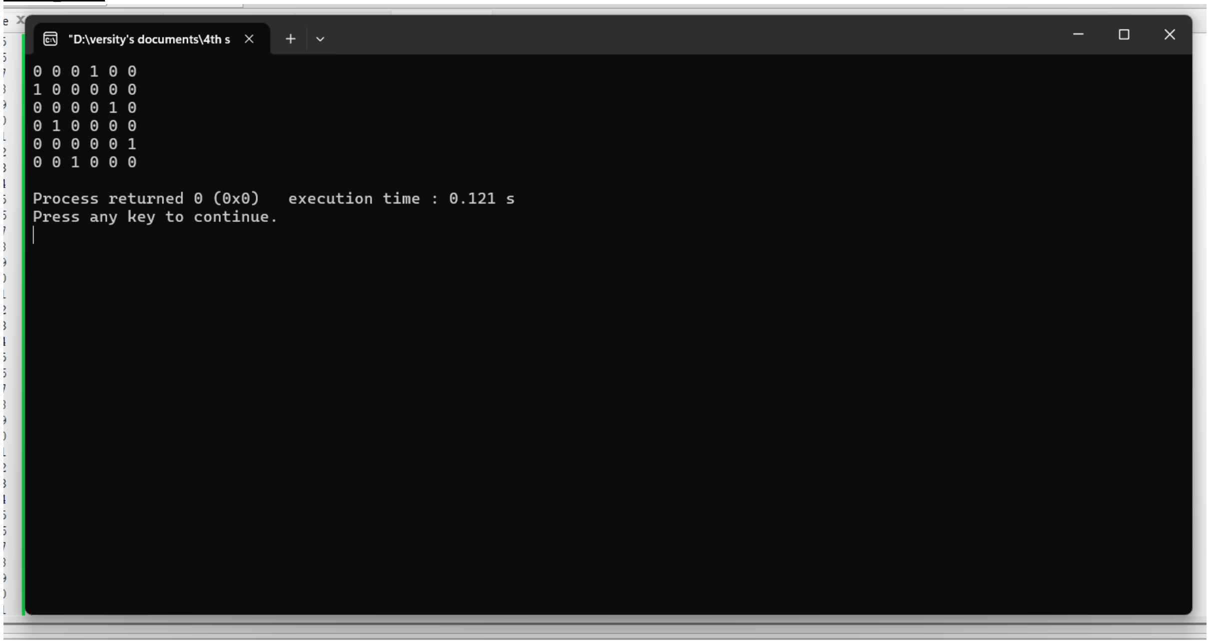
Demonstrate N-Queens problem using Backtracking:

Code:

```
#include <bits/stdc++.h>

using namespace std;

#include <ext/pb_ds/assoc_container.hpp>

using namespace __gnu_pbds;

using ll=long long;

using ull=unsigned long long;

using ld=long double;

const ll MOD=998244353;

const ll MAX_SIZE=200005;

template<typename T>

using ordered_set=tree<T,null_type,less<T>,rb_tree_tag,tree_order_statistics_node_update>;

bool canPlaceQueen(ll n,vector<vector<ll>>& grid,ll col)

{

    if(col>=n)return true;

    for(ll row=0; row<n; row++)

    {

        bool isValid=true;

        for(ll i=0; i<col; i++)if(grid[row][i])isValid=false;

        for(ll i=col,j=row; i>=0&&j>=0; i--,j--)if(grid[j][i])isValid=false;

        for(ll i=col,j=row; i>=0&&j<n; i--,j++)if(grid[j][i])isValid=false;

        if(isValid)

        {

            grid[row][col]=1;

            if(canPlaceQueen(n,grid,col+1))return true;

            grid[row][col]=0;

        }

    }

    return false;

}
```

```
void solveNQueens()
{
    ll n=6;
    vector<vector<ll>>grid(n,vector<ll>(n,0));
    if(!canPlaceQueen(n,grid,0))
    {
        cout<<"No valid configuration found\n";
        return;
    }
    for(ll i=0; i<n; i++)
    {
        for(ll j=0; j<n; j++)cout<<grid[i][j]<<" ";
        cout<<endl;
    }
}
int main()
{
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);
    solveNQueens();
    return 0;
}
```

Output:



<u>End</u>